CITS3402/CITS5507 Assignment 1: Parallel Hash Collision Attack

[Your Name], [Your Student Number]

Semester 2, 2026

Birthday-attack algorithm and collision-detection data structure

`toy_hash` produces a 48-bit output, so by the birthday bound a matching pair between two independent families of nonce trials is expected after roughly `sqrt(pi/2 * 2^48) ~ 1.25 * 2^24 ~ 2.1e7` combined trials — far fewer than the 2^48 a preimage search would need. The program exploits this directly: rather than fixing one file's nonce and searching only the other (a much larger expected search), it grows two pools of trials in parallel, one per file, and stops as soon as any trial's hash already exists in the *other* file's pool.

Both pools live in a single shared open-addressing hash table keyed by the 48-bit hash value. Each slot stores the hash, the nonce that produced it, which file it came from, and a "ready" flag. On insertion the hash is mixed with a Fibonacci multiplier and taken modulo the (power-of-two) table size to pick a home slot, then linear probing finds the first free or matching slot. If the slot already holds the same hash from the *other* file, a genuine collision has been found; if it holds the same hash from the *same* file, the trial is simply a redundant self-collision and is discarded. Table capacity is fixed at four times the trial budget (`4 * 2^26` slots by default), keeping the load factor at or below 0.5 so probe chains stay short even in the worst case. Nonces are generated sequentially (0, 1, 2, …) via an atomic counter per file rather than randomly — `toy_hash`'s MurmurHash3-style finalising mix already scatters sequential inputs uniformly across the 48-bit output space, so sequential generation gets the same statistical behaviour as random sampling while guaranteeing no nonce is ever wasted on a repeat trial.

Parallelisation and synchronisation with OpenMP

Threads are split evenly into an "A-group" and a "B-group" (even/odd thread id) inside one `#pragma omp parallel` region; each thread only ever generates and hashes trials for its assigned file, drawing the next unused nonce with `atomic_fetch_add` on a per-file counter. This gives lock-free, contention-free work distribution: no two threads ever hash the same nonce, and no thread ever blocks waiting for work.

The shared hash table is the only structure threads communicate through, and it is made safe without any `critical` section or `omp_lock_t`. Each slot's hash field is a C11 `_Atomic uint64_t` written only via `atomic_compare_exchange_strong` from `TABLE_EMPTY` to the trial's hash; exactly one thread can ever win that CAS for a given slot, so the `nonce/owner` fields belonging to that slot are subsequently written by that thread alone — no two threads ever write the same memory. The remaining hazard is a reader (another thread probing the same slot because it hashed to the same index) observing the slot as claimed before the winner has finished writing `nonce/owner`. This is closed with a small "published" flag: the winner writes `nonce/owner` first, then stores

`ready = 1` with `memory_order_release`; a thread that finds a matching hash spins on `ready` with `memory_order_acquire` before reading `nonce/owner`. The acquire/release pairing guarantees the reader never sees a torn write, and the spin window is only ever a few instructions long (between the CAS succeeding and the two plain stores that follow it), so it costs nothing in practice. A second, separate atomic flag (`found_flag`) is claimed with one final CAS so that only the first thread to discover a genuine cross-file collision records the winning nonce pair; every other thread simply observes the shared `stop_flag` at the top of its loop and exits. Before any file is written, the program independently recomputes `toy_hash` on the final modified bytes on the main thread and aborts if the two hashes disagree, so a bug in the concurrent path can never silently produce an invalid submission.

Memory requirements and trade-offs

The dominant cost is the collision table: at the default 2^26-trials-per-side budget, capacity is 2^28 slots, each holding an 8-byte hash, 8-byte nonce, and two 1-byte tags — about 4.3 GB. This was chosen deliberately over a smaller, higher-load-factor table: linear probing degrades sharply as load factor approaches 1, and Kaya nodes have far more memory than a table this size requires, so the trade favours search speed over memory economy. A second, much smaller cost is per-thread: each thread keeps one private, mutable copy of whichever file(s) it hashes (loaded once, header fields patched every trial), so only the 16-byte nonce field is rewritten per trial instead of re-copying the whole file — for the largest (~900 KB) pair with 96 threads this is under 90 MB, negligible next to the table.

Performance metrics and analysis

`search_seconds` measured on a 10-core Apple-silicon development machine (all trials verified with an independent hash, and against the reference `check_toy_hash.py`) for the smallest pair confirms both correctness and near-linear early scaling:

threads	search_seconds	speedup
1	21.53	1.00x
2	10.69	2.02x
4	6.00	3.59x
8	3.78	5.69x
10	3.42	6.29x

Both runs found the same ~10.39M trials on each side (~20.8M combined), matching the birthday-bound estimate above almost exactly. Efficiency drops from 100% at 2 threads to ~63% at 10 threads; the search is a memory-bandwidth-bound workload (every insert/probe is an effectively random access into a multi-gigabyte table), so once all cores are active they contend for shared cache and DRAM bandwidth rather than for compute, which is the expected shape of the curve and is expected to continue on Kaya's higher core counts. The 64 KB `1_kilo` pair was solved end-to-end at 10 threads in 182.7 s (10.12M trials/side), consistent with per-trial cost scaling with file size while the *number* of trials needed stays roughly constant across pairs — this is exactly why the harder, larger pairs need Kaya's full 96-core node rather than more trials.

[Kaya results to insert here after running `scripts/scaling_job.slurm` and `scripts/solve_all.slurm`: a table of `search_seconds` for all six pairs (`1_kilo` … `6_exa`) across thread counts 1–96, confirming

every pair solves within the 900-second budget, plus the resulting speedup and parallel-efficiency curves per pair.]